A MetaArchitecture for Distributed Resource Management *

Nalini Venkatasubramanian†
University of Illinois, Urbana-Champaign
Urbana, IL 61801

Carolyn Talcott
Stanford University
Stanford, CA 94305

Abstract

This paper presents a two-level model of distributed computation designed to provide a formal basis for specifying and reasoning about dynamic system modification and resource management activities in a distributed system. We take actors as our underlying computation model. An important role of such a model is to represent interactions among resource management activities and between these system level activities and application level activities. This allows us to express and reason about properties such as non-interference and composability. We illustrate the use of our model to specify resource management activities such as garbage collection and dynamic insertion of dependability protocols.

1 Introduction

One challenge of parallel distributed computation today lies in the fact that software technologies have not scaled up in proportion to hardware advances. An important issue for distributed computation is *openness*—the ability to interact with the environment, and to extend or modify parts of a system without stopping the system or interfering with other parts of the system. Another important issue is *composability* – the ability to combine existing program modules in different ways to achieve different functionalities. In this paper we present a two-level model of distributed computation that represents application and runtime activities in a uniform framework. This model addresses the issues of openness and composability by providing mechanisms for dynamic modification of these systems and by establishing a formal basis for specifying and reasoning about resource management activities, system modifications, and their interactions with each other and with the application level. We take as our computation model the actor model. We separate computation activities into application and system level, and correspondingly stratify actors. Base actors execute application programs. Meta actors can examine and modify the runtime data structures used in controlling execution of application programs. This model provides mechanisms for implementing specific system policies in the runtime system. It allows us to customize dynamic aspects of the system. Meta-level facilities are especially useful in a distributed system as it allows the designer to control and reason about aspects of concurrency like scheduling, communication and load balancing, and memory management, in a semantically clean manner. The programmer view at both levels is the same, the difference is the data structures being manipulated.

The two-level architecture naturally extends to multiple levels, with each level manipulating the level below (and protected from manipulation by lower levels). In practice, however expressing a computation in terms of multiple meta-levels becomes unwieldy. An alternative to providing multiple meta-levels is to move to a reflective architecture. Here the runtime system is presented as an actor system (meta interpreter), and access to the runtime structures is provided by reification and reflection. A reflective architecture allows one to provide an unbounded number of meta-levels with a single basic mechanism. Because of its power, reflection is more difficult to control and reason

*This research was partially supported by DARPA contract NAG2-703, by DARPA and NSF joint contract CCR 90-07195, by ONR contract N00014-90-J-1899, and by the Digital Equipment Corporation.

†Current address: Hewlett Packard MS44UT Cupertino, Ca 95014

0-8186-1060-3425/93 $03.00 © 1993 IEEE

Authorized licensed use limited to: Access paid by The UC Irvine Libraries. Downloaded on July 21,2024 at 15:04:40 UTC from IEEE Xplore. Restrictions apply.

about. Thus we feel that it is important to understand the simple two-level architecture before moving to the more powerful mechanism.

The remainder of this paper is organized as follows. In Section 2. we briefly review the actor model of computation. In Section 3. we present a two-level actor model as distributed transition system. In Sections 4–5. we illustrate the use of this model to specify and reason about two examples taken from existing literature: a distributed garbage collection algorithm, and methods for dynamic addition of dependability layers to applications. Section 7. contains discussion of further issues and future work.

2 The Actor Model

The *actor model*, first proposed by Hewitt [6] and later developed by Agha [1], captures the essence of concurrent computation in distributed systems at an abstract level. The actor model is a convenient perspective for the programmer of concurrent and distributed applications, due to its flexibility, simplicity, and ease of programming high level abstractions [2].

In the actor model, the universe contains computational agents called *actors*. Each actor has a *mail address* and a *behavior*. The mail address (sometimes called a handle or identifier) is conceptual rather than a physical location. The only way one actor can influence the actions of another actor is to send the latter a communication. An actor can send another actor (or itself) a communication only if it knows the mail address of the recipient. On receiving a communication, an actor processes the message in a manner determined by its current behavior. As a result the actor may: (1) create new actors; (2) change its behavior; and (3) send messages to existing actors. A formal distributed transition system semantics for a combined functional and actor language is presented in [4].

Communication in the actor model is *asynchronous* and *buffered*. In asynchronous communication, the sender and receiver do not need to coordinate message delivery. Every actor is equipped with a mailbox that queues any communications received. The mailbox serves as a buffer and provides a mechanism for serializing the processing of incoming communications. Another property of communication in the actor model is *guarantee of delivery*, i.e., messages in the system will eventually reach their destination actor. This implies that in-transit mail cannot be indefinitely delayed. The order of arrival of messages at a given actor imposes a notion of local time. The activation order (one message activates another if the latter is sent during the processing of the former) determines communication patterns. The combined ordering determines the possible dependencies among events in an actor system. Although the actor model is inherently asynchronous, it is possible to simulate synchronous models using activation dependencies to force suitable communication patterns.

Two important features of the actor model are the ability to create new actors, and the ability to communicate mail addresses. Thus the interconnection topology of the system is capable of changing continuously. The asynchrony and nondeterminism of arrival ordering together with use of conceptual rather than physical address allows resource management decisions such as scheduling, grain size, and object to processor mapping to be made dynamically.

3 The Two Level Actor Model

In order to account for the resource management activities going on behind the scene we introduce, meta-level actors. This provides a mechanism for programming these activities, and for reasoning about them within the actor model. Each actor in a system is either a base (application) actor or meta (system) actor. Base actors are actors that are created as a result of the execution of the application program. Meta actors are part of the resource management system. In general, there can be many kinds of meta actors in the actor system to handle various resource management duties. These actors communicate with each other via message passing, but they manipulate base actors as data structures. In particular a meta actor can access and modify information within a base actor, such as behavior, mail queue, and runtime tags, that other base actors cannot access. We also provide a mechanism for meta-actors to be notified when events of interest to them occur. This provides a more flexible mechanism for interaction of meta actors with the basic runtime system. It also provides a clean way for implementing daemons, monitors and similar features.

To represent resource management activities it is necessary to refine the basic actor model to make

Authorized licensed use limited to: Access paid by The UC Irvine Libraries. Downloaded on July 21,2024 at 15:04:40 UTC from IEEE Xplore. Restrictions apply.

explicit aspects such as the underlying network topology and the mapping of processes to processors. For this purpose we introduce the notion of a *multicomputer system*. It is a distributed system consisting of a network of autonomous processing units that communicate via message passing. The processing units do not physically share a common address space, and they have no globally synchronized clocks. Processing units are connected by unidirectional communication links along which messages are transmitted. The network is thus a directed graph whose nodes are the processing units and whose edges are the communication links. We assume that this graph is fully connected, i.e. every node is reachable from every other node via some sequence of links. We further assume that communication links behave as unbounded buffers with finite message transfer delay.

Although there is no global clock in a truly distributed system, we can assume that a meaningful notion of global state of the system exists (cf. [5]). We call such a global state a *multicomputer system configuration*. It is a snapshot of the entire state of the system, relative to some external point of view, that encompasses both the processing and communication activities in the system. It accounts for both the abstract level of processing and message passing and the distribution of this activity onto processing units and communication links. At the abstract level there is a finite number of base and meta actors, each with unique identity (its conceptual mail address). Each actor has an internal configuration representing its current behavior, local mail queue, and additional system defined attributes. In addition there are a finite number of messages in transit. To obtain a multicomputer system configuration we specify the mapping of actors (base and meta) onto the set of processing units, and the mapping of messages in transit to links and node buffers. We may specify, on a per node basis, additional structures such as a task pool, to expose more information about the runtime system of a node.

To describe the two-level actor model more formally, we assume given a network consisting of a set, **Node**, of nodes and a set, **Link**, of communication links. Each link has a source and target node and we assume that the network graph is completely connected. To describe the two level actor system we assume given countable sets: **Adr**, of actor conceptual addresses; **Loc**, of actor locations (physical addresses); **Tag**, of names for annotation fields; **Val**, of structured data constructed from some set of atoms and actor addresses; and **Beh**, of base/meta actor behavior codes. We define the following structures: **Msg** (base/meta messages), **Ic** (base/meta actor internal configurations), and **Note**, notifications. We use subscripts b/m to indicate the base/meta versions of the various entities. We assume that the set of conceptual addresses is partitioned into base actor addresses, $\mathbf{Adr}_b$, and meta actor addresses, $\mathbf{Adr}_m$. Thus $\mathbf{Adr} = \mathbf{Adr}_b \cup \mathbf{Adr}_m$.

A message has fields `target` : **Adr**, `content` : **Val** and possibly other annotation fields. An actor internal configuration has fields `beh` : **Beh**[**Val**], `mailq` : **Msg***, and possibly other annotation fields. A behavior *beh*[*val*] is a closure like object containing a behavior code and a sequence of values to be bound to the parameters of the behavior code. A notification is a pair consisting of an event type and a meta actor address. An event type specifies a set of message structures. We let **Evtype** denote the set of event types. For the present we do not further specify the structure of event types. This is summarized as follows (where subscripts $x \in \{b, m\}$ indicate base and meta level versions).

$\mathbf{Msg}_x$ = <`target` : $\mathbf{Adr}_x$, `content` : $\mathbf{Val}_x$, ...>
$\mathbf{Ic}_x$ = <`beh` : $\mathbf{Beh}_x[\mathbf{Val}_x]$, `mailq` : $\mathbf{Msg}_x^*$, ...>
Note = <**Evtype**, $\mathbf{Adr}_m$>

Using these structures we define the components of a two-level configuration: the name table, the base/meta actor configurations, the node configuration, and the communication configuration. The name table `nametable` is a finite map from conceptual addresses of actors, **Adr**, to physical locations, **Loc**. A location determines a node and a local address within that node (which may be stored in a local name table). In general the name table may contain additional information about an actor that allows for optimization of memory management, migration, and other operations. The base/meta actor configuration, $\mathtt{actcf}_{b/m}$, is a finite map from base/meta actor addresses to base/meta internal configurations. The node configuration, `nodecf`, is a finite map from nodes to internal node configurations. An internal node configuration consists

Authorized licensed use limited to: Access paid by The UC Irvine Libraries. Downloaded on July 21,2024 at 15:04:40 UTC from IEEE Xplore. Restrictions apply.

of several fields including: **notes**, the current notifications requested (a finite set of notifications), and **tasks**, the current task pool. The task pool consists of the set of base actors waiting to execute, and an ordering that determines which actor will execute next. For simplicity of exposition we represent the task pool as a sequence. We assume that additional information needed for task management is kept in annotation fields. The task pool contains only base level actors because we are making explicit only that part of the runtime system that meta-actors are allowed to manipulate. The communication configuration, **comcf**, has two fields: **nodeq**, a map from nodes to sequences of messages (the nodes mailque), and **linkq**, a map from links to sequences of messages. Thus a two level configuration is a structure with the following fields (where $[A \xrightarrow{\omega} B]$ is the set of finite maps from A to B).

$\mathbf{nametable} : [\mathbf{Adr} \xrightarrow{\omega} \mathbf{Loc}]$

$\mathbf{actcf}_x : [\mathbf{Adr}_x \xrightarrow{\omega} \mathbf{Ic}_x]$

$\mathbf{nodecf} : \mathbf{Node} \xrightarrow{\omega} \langle \mathbf{notes} : \mathcal{P}_\omega(\mathbf{Note}),\ \mathbf{tasks} : \mathbf{Adr}_b^* \rangle$

$\mathbf{comcf} : \langle \mathbf{nodeq} : [\mathbf{Node} \xrightarrow{\omega} \mathbf{Msg}^*],\ \mathbf{linkq} : [\mathbf{Link} \xrightarrow{\omega} \mathbf{Msg}^*] \rangle$

For coherency, we require configurations to satisfy the following conditions: (1) the domain of the name table is the union of the domains of the base and meta actor configuration maps; and (2) meta and base actors mentioned in messages in the communication configuration, or in internal configurations of base or meta actors are included in the domain of the name table.

A computation of a distributed system is a sequence of configurations (snapshots of the global state) that provide a consistent view of the evolution of the system over time. Formally, the possible computations of a two-level actor system are determined by the transition relation on configurations, together with constraints on the allowed computation paths to ensure mail delivery and other forms of fairness. For some applications additional requirements of the network topology, routing, and scheduling may be made. Corresponding to the three main components of a configuration, there are three kinds of transitions: communication, base, and meta. The communication transitions are determined by the network routing mechanism. **Communication Transitions:** There are three kinds of communication transition: (1) node queue to actor queue (local delivery); (2) node queue to link (routing); and (3) link to node queue.

(1) if the first message in the mail queue of a node ν, is for a local actor then it is removed from the nodes mail queue and added to the actors mail queue. If a notification is requested for this type of message, then a notification is added to the mail queue of each requesting meta actor.

(2) if the first message in the mail queue of a node ν, is for a non-local actor then the message is removed from the nodes mail queue and added to the queue of the link along which the message is to be routed.

(3) if a link l has a non-empty message queue then the message is removed from the links queue and added to the mail queue of the target node of the link.

The base and meta level transitions are determined by an interpretation, $\mathcal{E}$, of the corresponding behavior codes. $\mathcal{E}_b$ takes a base actor internal configuration and produces a new internal configuration, a set of new (base) messages to be sent, and a set of newly created base actors. $\mathcal{E}_m$ takes a meta actor internal configuration and a data structure representing the part of the configuration local to the node on which the meta actor resides: the base actor internal configurations for base actors on that node, and the nodes internal configuration. It also produces a new internal configuration, a set of new messages to be sent, and a set of newly created meta actors. It addition, it may produce updates to the local configuration information. For this paper we will not further specify the nature of behavior codes and their interpretation functions. A concrete example of base level codes and their transition semantics is presented in [4].

Base Transitions: Let a be a base actor address located on node ν such that a is the next actor to execute in the task pool of ν. Then the base transition for a modifies the configuration as follows. $\mathcal{E}_b$ applied to the current internal configuration for a. The new internal configuration for a is that produced by $\mathcal{E}_b$. The newly created actors are located on ν – they are entered in the name table, and added to the base actor internal configuration map. The new messages to be sent are added to the mail queue of ν. a is removed from the task pool, and one or more actors with messages pend-

Authorized licensed use limited to: Access paid by The UC Irvine Libraries. Downloaded on July 21,2024 at 15:04:40 UTC from IEEE Xplore. Restrictions apply.

ing, including possibly a, maybe added to the task pool.

Meta Transitions: Let a be the address of a meta actor located on node ν that is ready to execute. Then the meta transition for a modifies the configuration as follows. $\mathcal{E}_m$ applied to the current internal configuration for a, together with the base actor configuration map restricted to base actors on ν, the internal configuration of ν. As for the base level case, the new internal configuration for a is that produced by $\mathcal{E}_m$. The newly created meta actors are located on ν – they are entered in the name table, and added to the meta actor internal configuration map. The new messages to be sent are added to the mail queue of ν. In addition the base actor configuration and the internal node configuration of node ν is modified according to the updates produced by $\mathcal{E}_m$.

To fully realize the actor model, message routing and process scheduling must be *fair*, i.e. guarantee of mail delivery must be assured and a process with an allowed transition must eventually be scheduled. Thus a possible computation starting from a given configuration is sequence of configurations, with initial element being the given configuration, and with the succeeding elements being obtained by applying an allowed transition. We further require that either the sequence is finite with the final element having no allowed transitions, or that it is infinite and fair. To insure that the presence of meta-level actors does not destroy the integrity/semantics of the base-level system it is necessary to maintain a clean boundary between base and meta activities, and to enforce suitable constraints on the effects of meta activities. As a first example of how our two-level architecture can be used, we describe how to express the key property that the meta-level computations do not interfere semantically with the base-level computations. For this purpose, we define the notion of restriction, $C \downarrow b$, of a configuration C to base level. This is the configuration obtained by: (1) deleting meta actor addresses from the name table; (2) replacing the meta actor configuration map by the map with empty domain; (3) erasing annotation fields from base actors and messages; (4) replacing the notifications on each node by the empty set; and (5) deleting any messages to meta actors in the communication configuration. Now, we say that the computations from a given configuration C do not interfere with the base level if for any computation of $C \downarrow b$ there is a computation of C whose base level restriction is that computation. This is a very strong notion of non-interference. Other useful notions can be obtained by considered equivalence classes of computations (cf. [9]). For example computations that differ only by permuting the execution of independent transitions can be considered equivalent.

In the remainder of this paper we illustrate the use of the two-level actor model to specify two different resource management activities: garbage collection, and replication for dependability. We consider examples taken from the existing literature. The particular choices made by the designers of these resource managers are not of concern here. The point is to show how such activities can be expressed as particular two-level actor machines. To describe a resource management activity, we must specify its meta actors, and the annotation fields of base actors and messages that they manipulate.

4 Distributed garbage collection

The Hierarchical Distributed Garbage Collection (HDGC) algorithm [11, 12] is a non-halting garbage collection algorithm for active objects distributed across a network of systems. In this section we show how to formalize HDGC as a two-level actor machine. We begin with a summary of the algorithm. We then describe the GC annotations and the roles of the GC meta actors. Finally we indicate how properties of the algorithm needed to express and prove correctness can formalized in terms of the two-level actor machine.

Hierarchical Distributed Garbage Collection: Hierarchical organization provides for scalability. It partitions a distributed system into smaller subsystems, which in turn may be further partitioned. The topmost level of the hierarchy is the entire system and the lowermost level of the hierarchy has a single node per subsystem. The definition of reachable and garbage objects in an actor-based system is derived from the work of Kafura et al [7]. The *root set* is a fixed set of actors from which reachability is traced. At a given point in time, the *acquaintances* of an actor are those actors whose address is 'known' by that actor, i.e. addresses occurring in the behavior of the actor or in messages sent to the actor but not yet pro-

Authorized licensed use limited to: Access paid by The UC Irvine Libraries. Downloaded on July 21,2024 at 15:04:40 UTC from IEEE Xplore. Restrictions apply.

cessed. The *inverse acquaintances* of an actor are those actors who know its address.

In an actor computation, the transitive closure of the acquaintance relation starting from the root set is not adequate to determine reachability. This is due to the fact that in systems of active objects, such as actors, an apparently unreachable object may at some point communicate its mail address to a reachable acquaintance thereby becoming reachable. Thus inverse acquaintances must also be considered in determining reachability. An actor is *active* if it is currently processing a message, or if it has messages pending in the network or in its mail queue. An actor that is neither active nor connected by the transitive closure of the inverse acquaintance relation to an active actor is *permanently inactive*. An actor that is permanently inactive can never communicate its mail address and can be safely regarded as unreachable. The set of *reachable* actors is defined inductively as the least set such that, (1)Every actor in the root set is reachable; (2)Every acquaintance of a reachable actor is reachable and (3)If an actor is reachable, then every inverse acquaintance of that actor which is not permanently inactive is reachable. An actor that is not reachable according to the above definition is *garbage* and resources allocated to this actor can be reclaimed.

The *distributed scavenging* algorithm for marking the reachable objects in the system follows the inductive definition of reachability. To record the reachable objects, each object of the system has associated with it an *object-status* which may be *touched*, *untouched* or *suspended*. Touched objects are objects which are known to be reachable. Untouched objects have not yet been visited during GC. Objects that remain untouched at the completion of GC are unreachable. Suspended objects are inactive objects that are inverse acquaintances of reachable (touched) objects. If an active inverse acquaintance of such an object is found then the object will become touched. An object that remains suspended at the completion of GC is also unreachable. When GC is initiated all actors in the system have status untouched. Any actors created after the start of GC on a node are marked as touched. The marking of objects is accomplished by propagation of GC and GC^{-1} messages from the roots and by backpropagation of GCack and GC^{-1}ack messages.

It remains to explain how to compute the acquaintance relation and the activity status of each base actor. These are properties that depend on global state of the system. In a system with distributed state there is no uniquely determined global state. Thus to compute some property of the state it is generally necessary to record a consistent global snapshot of the information needed. The HDGC algorithm uses multicast protocols for propagation of messages to obtain a consistent snapshot of the acquaintance and active status information. We assume that the nodes in the network topology are ordered, such that it is possible to designate a *start* node and a *finish* node. The multi-cast messages used are of two kinds: *broadcast messages* and *bulldoze messages*. There are two types of bulldoze messages, *forward* and *backward*, every node has a set of forward bulldoze neighbors and a set of backward bulldoze neighbors. *Forward bulldoze messages* are initiated at the start node and propagate from every node to its respective forward neighbors. *Backward bulldoze messages* are initiated at the finish node and propagate from every node to its respective backward neighbors. Bulldoze messages traverse every pair of links in the network and, by the FIFO assumption on links, force messages already in the network to be cleared along the direction of the bulldoze. In the case of broadcast messages, every node has a set of broadcast neighbors. *Broadcast messages* are initiated at the start node and propagate from every node to its respective broadcast neighbors. Broadcast messages reach every node in the system, but traverse only a subset of the links and hence are less costly.

We present the HDGC Algorithm in the context of a two level hierarchy, i.e. global and node level collections. The generalization to hierarchies with intermediate levels is straightforward. Receptionist tables [11] maintain accessibility information at any level in the hierarchy. We can use the reachability information at lower levels in the hierachy to perform GC on the next higher level in the hierarchy, thereby allowing a scalable GC mechanism. We can use any of the traditional algorithms for local GC. The HDGC algorithm consists of five steps: Pre-GC, Distributed-Scavenge, Local-Clear Initiation, Local-Clear, and Post-GC. The GC steps are

Authorized licensed use limited to: Access paid by The UC Irvine Libraries. Downloaded on July 21,2024 at 15:04:40 UTC from IEEE Xplore. Restrictions apply.

initiated and carried out by communication of GC related messages. We describe the purpose of each step below.

Step 1: Pre-GC. During the pre-GC step each node is notified that a GC has been initiated via a preGC broadcast. This defines a local start-of-GC time on each node that is globally consistent. The network is cleared of *old* messages (messages in the system prior to initiation of GC) using a forwards bulldoze followed by a backwards bulldoze. During this period each node records acquaintances of its base actors relative to start-of-GC time. A second forwards/backwards bulldoze pass is used to record inverse acquaintances locally. The GC annotations will persist throughout the duration of the GC. The combined local information forms a consistent global snapshot of the system state that adequately records reachability information about each actor in the system. We call this the *GC snapshot*.

Step 2: The Distributed Scavenge. During this step, actors that are non-garbage relative to the GC snapshot are marked *touched* using the *distributed scavenge algorithm* described above.

Step 3: Local Clear Initiation. During this step, each node in the system is informed that the distributed scavenge phase has completed and local clearance begins. On each node, objects not marked touched are cleared from local memory, according the nodes method of memory management, and any other actions (updating tables, etc.) entailed by this reclamation are carried out.

Step 4: The Local Clear. This step detects when all nodes have completed the local collection initiated in the previous step.

Step 5: Post GC. During this step each node is informed that the current GC is complete: each node can now note that GC is no longer in progress and update necessary information to reflect this state. At the end of this step a new GC can be initiated at anytime.

4.1 HDGC in the Two Level Model

Each node has a single GC meta actor, and there is a unique (per subsystem) special actor designated as the GC-root actor. Requests to initiate a GC go to the GC-root actor and sequencing of the GC steps are synchronized through the GC-root actor. Each GC meta actor has internal states corresponding to each stage of GC and handles the basic GC meta messages for carrying out each of the steps. The GC meta actors residing on the start and finish nodes have additional work to do for synchronization. In some cases they must notify the GC-root when a multicast message has propagated through the system, in other cases they must initiate another multicast. Four GC fields are added to base actor internal configurations: `GC-acq`: $\mathcal{P}_\omega(\mathbf{Adr}_b)$; `GC-iacq`: $\mathcal{P}_\omega(\mathbf{Adr}_b)$; `GC-status`: {`touched`, `untouched`, `suspended`}; and `GC-activity`: {`active`, `inactive`}. One GC field is added to base messages: `GC-tag`: {`old`, `new`}. The `GC-acq` and `GC-iacq` fields are set by the GC meta actor during the preGC step. The `GC-activity` field is set by the GC meta actor during the preGC step. The `GC-status` field is initialized as `touched` during GC and as `untouched` otherwise. The `GC-tag` field is initialized as `new` during GC and as `old` otherwise. A GC meta actor must inform the actor creation mechanism on its node of start and end of GC times. A GC meta actor requests to be notified of messages whose `GC-tag` field is `old` at the start of preGC step, in order to record acquaintances contained in these messages. When the network is cleared of old messages, then this notification request is removed.

Formalizing properties of HDGC: To illustrate the utility of the representation of the HDGC algorithm using GC meta actors in a formal specification process, we indicate how two properties of a multicomputer system with a GC subsystem can be expressed. These properties are in fact used in the statement and verification of correctness of HDGC. First we formalize global reachability relative to a given configuration.

$\mathbf{Acq}(C)$ is the acquaintance relation (on base actor addresses) relative to configuration C. (a, a') is in $\mathbf{Acq}(C)$ just if a' occurs in $C.\mathtt{actcf}_b(a)$ or a' occurs in a message to a that occurs in $C.\mathtt{comcf}$.

$\mathbf{Active}(C)$ is the set of active base actors relative to configuration C. a is in $\mathbf{Active}(C)$ just if the mail queue of a in C, $C.\mathtt{actcf}_b(a).\mathtt{mailq}$, is nonempty, or there is a message to a that occurs in $C.\mathtt{comcf}$.

$\mathbf{Reach}(C, r)$ is the set of actors reachable from root set r relative to C. $\mathbf{Reach}(C, r)$ is computed according to the inductive definition of reachability using $\mathbf{Acq}(C)$ for the acquaintance relation, and $\mathbf{Active}(C)$ for the active predicate.

Authorized licensed use limited to: Access paid by The UC Irvine Libraries. Downloaded on July 21,2024 at 15:04:40 UTC from IEEE Xplore. Restrictions apply.

Using the definition of reachability, we can formalize the property that an unreachable actor can never become reachable.

- If a is a base actor in C (is in the domain of C.`nametable`) and not in $\mathbf{Reach}(C, r)$ and $C \mapsto C'$ then a is not in $\mathbf{Reach}(C', r)$.

Next we define reachability according to the GC snapshot.

$\mathbf{GC\text{-}Acq}(C)$ is the acquaintance relation relative to the GC snapshot recorded in C. (a, a') is in $\mathbf{GC\text{-}Acq}(C)$ just if a' occurs in the GC acquaintances field of $C.\mathtt{actcf}_b(a)$.

$\mathbf{GC\text{-}iAcq}(C)$ is the inverse acquaintance relation relative to the GC snapshot recorded in C. (a, a') is in $\mathbf{GC\text{-}iAcq}(C)$ just if a' occurs in the GC inverse acquaintances field of $C.\mathtt{actcf}_b(a)$.

$\mathbf{GC\text{-}Active}(C)$ is the set of active actors relative to the GC snapshot recorded in C. a is in $\mathbf{GC\text{-}Active}(C)$ just if the GC active field of a is true.

$\mathbf{Reach}_{\mathrm{GC}}(C, r)$ is the set of base actors with `GC-status` field `touched` in C (this accounts for newly created actors) together with the set of base actors reachable from root set r computed according to the inductive definition of reachability using $\mathbf{GC\text{-}Acq}(C)$ for the acquaintance relation, $\mathbf{GC\text{-}iAcq}(C)$ for the inverse acquaintance relation, and $\mathbf{GC\text{-}Active}(C)$ for the active predicate. The second property says that the GC snapshot adequately represents reachability.

- During a GC initiated under suitable initial conditions, in any configuration C after the Pre-GC step is complete, if a is in $\mathbf{Reach}(C, r)$ then a is in $\mathbf{Reach}_{\mathrm{GC}}(C, r)$.

In summary, the HDGC example illustrates: the use of base actor and message annotations; communication between meta actors and the basic runtime system to insure correct initialization of annotations; and the use of the computation model to formalize properties of meta level activities.

5 Replication for dependability

Dependability or the ability to maintain availability of system facilities in spite of software or hardware failures is one of the important services in a distributed system. In this section, we represent a composable, application-independent mechanism to guard against fail-stop [10] failures in a system by using replication. The replication protocol modeled here has been described in [3], which introduces MAUD (Meta Architecture for Ultra Dependability).

A server is an actor system with a single receptionist that expects a request for the service along with a reply address. For the present we will assume that a server is not history sensitive (has no internal state). To increase dependability, a service can be replicated. To do this, several exact copies (replicas) of this service are created on different processing nodes in the system. Any request to the service is rerouted to all the replicas by a *distributor* and responses are intercepted by a *collector*. The first response received to a client (application actor's) request is returned and further responses from other replicas are ignored. In this way, there is a high probability that the client process will receive a response to its request even if one (or more) of the nodes fail. A replicated server is again a server (exactly one reply is sent in response to each request), and replication does not change the functionality of a service.

One way of modelling dependability via replication in the two-level model is to define a dependability subsystem in which there is a Replication Dependability MetaActor (RDMA) on each node. To replicate a service, a request is sent to an RDMA containing the address of the service actor to be replicated, and a list of nodes on which it is to be replicated. When such a request is received, the RDMA creates a collector actor C, and sends a request to the RDMA actors on the nodes in the list to create local replicas of the service using collector C. When the addresses of the requested local replicas have all been sent to the initiating RDMA, a distributor behavior is created and installed in place of the service behavior. The behaviors of the distributor, collector, and replicas are is described below.

Distributor: The Distributor (called serveMQ in [3]) has as acquaintances the list of replicas. It also has a generator of new identifiers. When a client request arrives, a new identifier is generated and a message containing the identifier and the original request is sent to each replica.

Collector: The collector (called serveDispatcher in [3]) maintains a log of replies received for each request, using the unique identifier associated

Authorized licensed use limited to: Access paid by The UC Irvine Libraries. Downloaded on July 21,2024 at 15:04:40 UTC from IEEE Xplore. Restrictions apply.

with each request. When a reply with identifier *id* is received from a replica, the collector looks to see if there is a previous reply to the corresponding request. If not, the identifier and reply are logged and the reply is sent on to the customer. Otherwise the reply is logged and discarded. Each entry thus contains the number of replica responses received. When all replicas have sent a response, the entry can be deleted from the response table.

Replica: A replica consists of three actors: an actor with the same behavior as the original server; a receiver; and a transmitter, that appears to the outside as a single actor whose address is that of the receiver. The receiver receives messages containing an identifier and a client request. When the server is ready for the next request, the receiver sends the identifier and client address to the transmitter and sends the request with the clients address replaced by the transmitter address to the server actor. The receiver then queues further requests from clients until the transmitter signals that the server has replied to the current request. When the transmitter receives a reply, it signals the receiver and sends a message containing the identifier, the reply, and the client address to the collector. Thus the receiver/transmitter behave as pre/post processors of messages to the server that make the request identifier transparent to the server.

The Replication Dependability subsystem represents a quite different use of the two-level architecture than the GC subsystem. It makes no use of annotation fields either in base level actors or in messages. It uses no notifications. On the otherhand an RDMA must be able to create new base level actors, send descriptions of base level actors to be created by other meta actors, and install new behaviors for existing base actors.

6 Conclusions and future work

In this paper, we present a two-level model of distributed computation based on the actor model. This model treats both application (base level) and runtime (meta level) computation activities in a uniform framework. To illustrate the basic ideas, we gave two examples: distributed garbage collection, and introduction of dependability protocol layers. The former shows how annotations and communication with the basic runtime system can be used to compute global properties of the base level system. and how the framework provides a basis for expressing a proving properties of such programs. The latter is an example of how this architecture allows dependability protocols and other runtime aspects of an application to be programmed independently of the basic application behavior, and how it provides for dynamic tuning of the runtime behavior. This independence allows a wide variety of protocols and aspects of execution behavior to be composed from a small number of basic modules. An example of this modular composition is the *layered* or onion-skin approach for combining dependability protocols described in [3]. The system that we have described assumes that the installation of of meta-objects is performed on the fly. This gives us the ability to add new features to an operating system without *halting* the system or *rebooting* – an essential characteristic of open systems.

We view this work as the first step towards development of a powerful tool for designing practical and robust operating systems for open distributed systems. This paper has focused on the structure of the underlying model. We regard annotations of base-level objects as extensible structures with different views provided for each meta-level activity. We have not yet developed a mechanism for introducing resource management activities, however some of the requirements of such a mechanism are clear. An activity has an associated set of annotations. Only the meta actors associated with that activity are allowed to directly manipulate these annotations. Introduction of an activity must specify to the basic runtime system how annotations are to be initialized. We are working on representing a variety resource management activities in the two level actor model: grain size control; task migration; debugging tools; event monitors; synchronization constraints, to mention a few.

Another aspect of this work is to develop systematic means of describing and ensuring constraints on interactions among meta level activities. Typically, while reasoning about resource management activities, we tend to assume independence of resource management activities from each other. But, in a real system resource management activities are not necessarily independent. In addition, interleaving the base-level and meta-level execution may give rise to inconsistencies if

Authorized licensed use limited to: Access paid by The UC Irvine Libraries. Downloaded on July 21,2024 at 15:04:40 UTC from IEEE Xplore. Restrictions apply.

improperly done. For example, since the replication subsystem neither reads nor writes any annotations, and new actors are accounted for by the allocation subsystem, the replication subsystem interacts non-trivially with GC only at the point that a new behavior for a server is installed.

Related Work: A number of object oriented concurrent reflective languages have been designed and implemented. ABCL/R and ACT/R [13] are languages that permit reflection.ABCL/R2 [8] is an OOCR language with a hybrid group reflective architecture that expresses the notion of shared resources and heterogeneous groups linguistically. Traditional reflective systems aim at providing a customizable and adaptable execution of parallel systems, albeit linguistically. Reflective systems are *evolutionary* systems; they form an effective prototyping testbed, a platform to test and evaluate various implementation policies. However, as we mentioned earlier, we believe that it is important to understand and embed a resource management system in a two level framework before trying to deal with the additional power of purely reflective systems. In order to effectively and correctly utilize of the power of reflection, it is necessary to understand all levels of the reflective architecture. It is not clear to us that giving the application programmer the freedom to alter the system behavior via reflection is desirable — customizable operating systems.

Acknowledgments. The authors whould like to thank Gul Agha for numerous conversations and ideas that played a key role in this work. They would also like to thank the anonymous referees for careful reading of an earlier version and for many suggestions for improvement of the presentation.

References

[1] G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems.* MIT Press, Cambridge, Mass., 1986.

[2] G. Agha. Concurrent object oriented programming. *Comm. of the ACM*, 33(9):125–141, 1990.

[3] G. Agha, S. Frolund, R. Panwar, and D. Sturman. A linguistic framework for dynamic composition of dependability protocols. In *Proceedings of the 3rd IFIP Working Conference on Dependable Computing for Critical Applications*, Palermo (Sicily), Italy, September 1992.

[4] G. Agha, I. Mason, S. Smith, and C. Talcott. Towards a theory of actor computation. In *Proceedings of the Third International Conference on Concurrency Theory, CONCUR 92*, August 1992.

[5] W. D. Clinger. Foundations of actor semantics. AI-TR- 633, MIT Artificial Intelligence Laboratory, May 1981.

[6] C. Hewitt. Viewing control structures as patterns of passing messages. *Journal of Artificial Intelligence*, 8-3:323–364, June 1977.

[7] D. Kafura, D. Washabaugh, and J. Nelson. Garbage collection of actorrs. In Norman Meyrowitz, editor, *1990 ECOOP/OOPSLA Proceedings*, pages 126–134, Ottawa, Canada, October 1990. ACM Press.

[8] S. Matsuoka, T. Watanabe, and A. Yonezawa. Hybrid group reflective architecture for object oriented concurrent reflective programming. *Lecture Notes in Computer Science*, 512:231–250, 1991.

[9] J. Meseguer. Rewriting as a unified model of concurrency. In *Proceedings of Concur'90*, volume 458 of *Lecture Notes in Computer Science*, pages 384–400. Springer-Verlag, 1990.

[10] F.B. Schneider. The state machine approach. *Lecture Notes in Computer Science*, 448:18–41, 1990.

[11] N. Venkatasubramanian. Hierarchical garbage collection in scalable distributed systems. Master's thesis, University of Illinois, Urbana-Champaign, Dept. of Computer Science, Urbana, IL, January 1992.

[12] N. Venkatasubramanian, G. Agha, and C. Talcott. Scalable distributed garbage collection for systems of active objects. In *Proceedings of the International Workshop on Memory Management 92*, Lecture Notes in Computer Science, September 1992.

[13] T. Watanabe and A. Yonezawa. An actor based metalevel architecture for group-wide reflection. In *Proceedings of the REX School/Workshop on Foundations of Object Oriented Languages, Lecture Notes in Computer Science*, May 1990.

Authorized licensed use limited to: Access paid by The UC Irvine Libraries. Downloaded on July 21,2024 at 15:04:40 UTC from IEEE Xplore. Restrictions apply.